

Ai Assisted coding

2303A510A0(Vishnu vardhan)

[Batch-14]

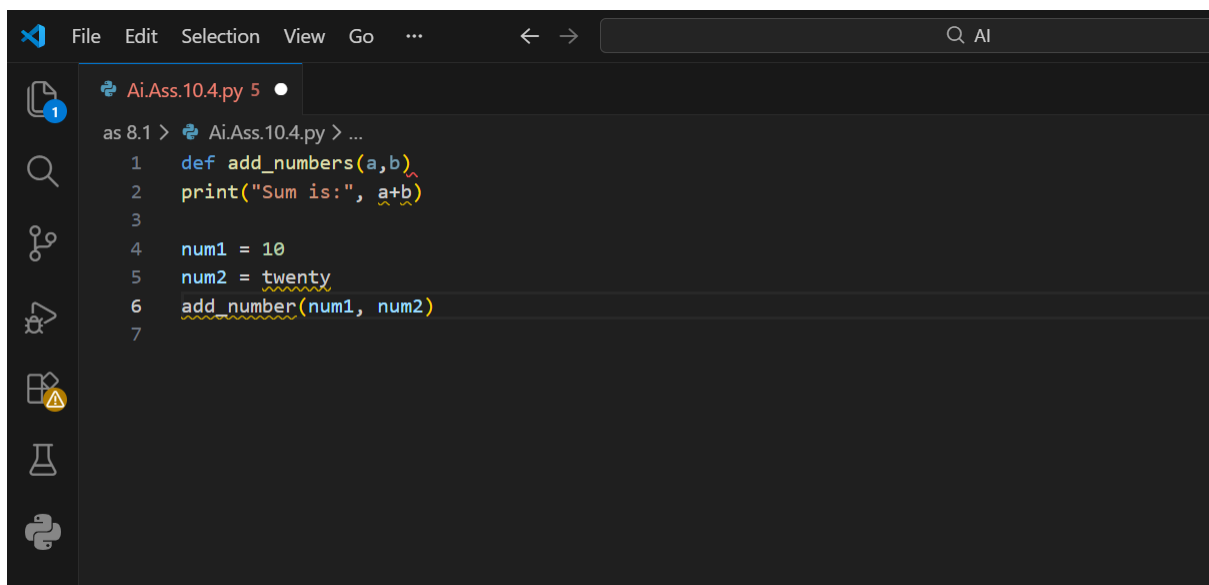
Assignment -10.4

#Code Review and Quality: Using AI to Improve Code Quality and Readability.

Task-1: AI-Assisted Syntax and Code Quality Review

Prompt: Identify all syntax errors, indentation issues, naming mistakes, and structural problems. Correct the code completely.

Explain each correction made. Make it clean and readable.



The screenshot shows a code editor window with a dark theme. The menu bar at the top includes 'File', 'Edit', 'Selection', 'View', 'Go', and a search bar with 'AI' entered. The file explorer on the left shows a file named 'Ai.Ass.10.4.py' with 5 lines. The code editor displays the following Python code:

```
as 8.1 > Ai.Ass.10.4.py > ...
1  def add_numbers(a,b)
2  print("Sum is:", a+b)
3
4  num1 = 10
5  num2 = twenty
6  add_number(num1, num2)
7
```

The code contains several syntax errors: a missing closing parenthesis in the function definition on line 1, a missing closing parenthesis in the print statement on line 2, an undefined variable 'twenty' on line 5, and a function call 'add_number' on line 6 that does not match the defined function 'add_numbers'.

```
def add_numbers(a,b)
print("Sum is:", a+b)
def add_numbers(a, b):
    print("Sum is:", a + b)

num1 = 10
num2 = twenty
add_number(num1, num2)
num2 = 20
add_numbers(num1, num2)
```

Detailed Explanation of Each Correction

1. Missing Colon (:) on Function Definition

- Error: [def add_numbers\(a,b\)](#)
- Why it's wrong: Python requires a colon at the end of function definitions to mark the start of the code block
- Corrected: [def add_numbers\(a, b\):](#)
- Impact: Without this, the code won't run at all (SyntaxError)

2. Missing Indentation in Function Body

- Error: [print\("Sum is:", a+b\)](#) was at the same level as the function definition
- Why it's wrong: In Python, code inside a function must be indented (typically 4 spaces). This tells Python which statements belong to the function
- Corrected: Indented the [print\(\)](#) statement with 4 spaces
- Impact: Without proper indentation, the print statement wouldn't be part of the function, causing a logical error

3. Invalid Variable Value

- Error: [num2 = twenty](#)

- Why it's wrong: twenty is treated as a variable name (not defined), not the number 20. Python would throw a NameError when it tries to find a variable called twenty
- Corrected: [num2 = 20](#)
- Impact: The code would crash with "NameError: name 'twenty' is not defined"

4. Function Name Mismatch

- Error: Function defined as [add_numbers\(\)](#) but called as add_number()
- Why it's wrong: The function name and the call must match exactly. Python can't find a function called add_number() because it's defined as [add_numbers\(\)](#)
- Corrected: Changed the call to [add_numbers\(num1, num2\)](#)
- Impact: Without this fix, you'd get "NameError: name 'add_number' is not defined"

5. Formatting Issues (Style)

- Missing spaces after comma: [\(a,b\)](#) → [\(a, b\)](#) - follows PEP 8 style guide
- Inconsistent spacing in expression: [a+b](#) → [a + b](#) - cleaner, more readable style.

Task-2: Performance-Oriented Code Review

Prompt: Analyze this function for performance bottlenecks. Refactor it to improve time complexity. Preserve correctness. Explain why the original code is inefficient and how the optimized version improves performance.

```
as 8.1 > task2.py > ...
1 def find_duplicates(lst):
2     duplicates = []
3     for i in range(len(lst)):
4         for j in range(i + 1, len(lst)):
5             if lst[i] == lst[j] and lst[i] not in duplicates:
6                 duplicates.append(lst[i])
7     return duplicates
8 numbers = [1, 2, 3, 4, 5, 2, 3, 6, 7, 8, 1]
9 print("Duplicates:", find_duplicates(numbers))
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS Python

PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python38-32/python.exe C:/Users/vishn/OneDrive/Desktop/AI/as 8.1/task2.py"

• Duplicates: [1, 2, 3]

○ PS C:\Users\vishn\OneDrive\Desktop\AI>

```
as 8.1 > task2.py > ...
1 def find_duplicates(lst):
2     duplicates = []
3     for i in range(len(lst)):
4         for j in range(i + 1, len(lst)):
5             if lst[i] == lst[j] and lst[i] not in duplicates:
6                 duplicates.append(lst[i])
7     return duplicates
2 seen = set()
3 duplicates = set()
4 for num in lst:
5     if num in seen:
6         duplicates.add(num)
7     else:
8         seen.add(num)
9     return list(duplicates)
10 numbers = [1, 2, 3, 4, 5, 2, 3, 6, 7, 8, 1]
11 print("Duplicates:", find_duplicates(numbers))
```

Detailed Explanation

Why the Original Approach Was Inefficient

- Nested Loop Complexity ($O(n^2)$ at minimum)

- The outer loop runs n times, and the inner loop runs up to n times for each outer iteration
- This creates a quadratic relationship—for 1,000 elements, you get ~500,000 comparisons just from the nested loops alone
- Expensive Membership Testing ($O(n)$ per check)
- The condition `lst[i] not in duplicates` searches the entire `duplicates` list linearly
- With lists, Python must check every element one-by-one to determine if an item exists
- This happens inside the nested loops, multiplying the cost: $O(n^2) \times O(n) = O(n^3)$ total
- Real-world Impact
 - For a 100-element list: ~1 million operations
 - For a 1,000-element list: ~1 billion operations
 - For a 10,000-element list: ~1 trillion operations
- The algorithm becomes unusable with even moderately sized data
- Redundant Comparisons
 - The algorithm compares the same pair of elements multiple times across different iterations
 - No early termination or optimization after finding a duplicate

How the Optimized Version Improves Performance

- **Single Linear Pass ($O(n)$)**
 - Instead of nested loops, the algorithm makes only one pass through the list
 - Each element is examined exactly once

- This reduces 500,000 comparisons on a 1,000-element list down to just 1,000 operations
- **Constant-Time Set Lookups ($O(1)$)**
 - Sets use hash tables internally for near-instant lookups
 - Checking `if num in seen` takes $O(1)$ constant time regardless of set size
 - Compared to lists where checking `if num in duplicates` requires scanning potentially all elements
- **Comparison: List vs Set Lookup**

- **Total Complexity Reduction**

- **Original:** $O(n)$ loop $\times$ $O(n)$ inner loop $\times$ $O(n)$ list lookup = $O(n^3)$
- **Optimized:** $O(n)$ loop $\times$ $O(1)$ set lookup = $O(n)$
- **Speed improvement:** For $n=1,000$: 1 billion ops $\rightarrow$ 1,000 ops
= **1,000,000x faster** 🚀

- **Memory Efficiency**

- Both use $O(n)$ space, but sets are more efficient storage than lists for this use case
- No wasted iteration or redundant data structure operations

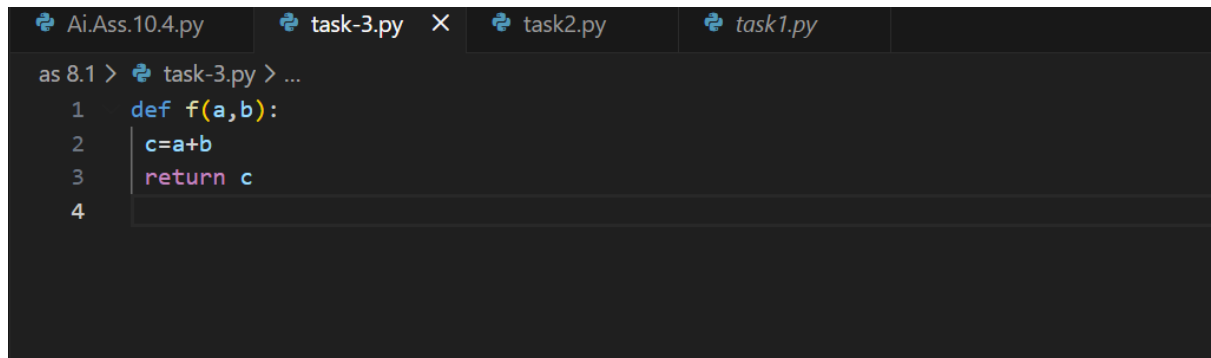
- **Scalability**

- Original fails on large datasets (becomes noticeably slow around 5,000-10,000 elements)
- Optimized handles 100,000+ elements in milliseconds

The key insight: **Replace costly nested loops with a single pass, and use the right data structure (set) for the job.**

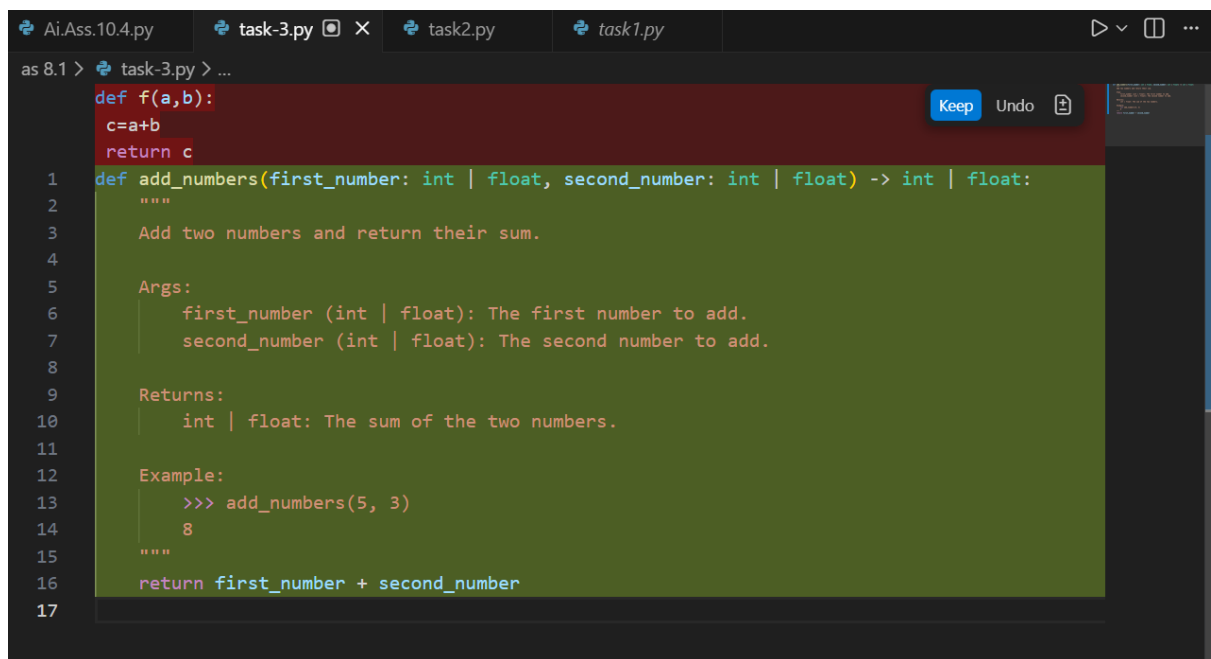
Task-3: Readability and Maintainability Refactoring

Prompt: Refactor this Python function for readability and maintainability. Apply PEP 8 standards. Improve naming conventions. Add meaningful docstrings. Explain the readability improvements.



The screenshot shows a code editor with four tabs: 'Ai.Ass.10.4.py', 'task-3.py', 'task2.py', and 'task1.py'. The 'task-3.py' tab is active, showing a Python function `f(a,b)` with three lines of code: `c=a+b` and `return c`. The prompt text is visible above the code.

```
as 8.1 > task-3.py > ...
1  def f(a,b):
2      c=a+b
3      return c
4
```



The screenshot shows the same code editor with the 'task-3.py' tab active. The original function `f(a,b)` has been replaced by a new function `add_numbers` with a comprehensive docstring. The docstring includes a description, arguments, returns, and an example. The function body is `return first_number + second_number`. A 'Keep' button is visible in the top right corner of the code block.

```
as 8.1 > task-3.py > ...
def f(a,b):
    c=a+b
    return c

1  def add_numbers(first_number: int | float, second_number: int | float) -> int | float:
2      """
3          Add two numbers and return their sum.
4
5          Args:
6              first_number (int | float): The first number to add.
7              second_number (int | float): The second number to add.
8
9          Returns:
10             int | float: The sum of the two numbers.
11
12         Example:
13             >>> add_numbers(5, 3)
14             8
15         """
16         return first_number + second_number
17
```

Key Improvements Explained

1. Function Name: `f` → [add_numbers](#)

- **Original problem:** `f` is meaningless—it could be any function

- **How it helps:** The name immediately communicates purpose. Anyone reading the code knows exactly what the function does without examining the implementation
- **Standard:** Follows PEP 8 naming convention for functions (lowercase with underscores)

2. Parameter Names: `a, b` → `first_number, second_number`

- **Original problem:** Single letters are fine in math, but in code they hide meaning
- **How it helps:** Clear names act as built-in documentation—you understand what each parameter represents without reading the function body
- **Semantic clarity:** Someone using this function knows they're passing numbers, not arbitrary values

3. Type Hints: `None` → `int | float`

- **Original problem:** No way to know what types the function accepts. You had to guess or read the code
- **How it helps:**
 - IDE shows autocomplete hints and catches type errors before runtime
 - Documentation is enforced at the code level
 - Makes debugging easier—type mismatches are caught early
- **Modern Python:** Union types (`int | float`) are clearer than the older `Optional[Union[...]]` syntax

4. Return Type Hint: Missing → `-> int | float`

- **Original problem:** Unknown what the function returns

- **How it helps:** Readers know the function returns either an int or float, enabling proper handling of the result

5. Comprehensive Docstring

- **Original problem:** Zero documentation. Users must reverse-engineer the function
- **How it helps:**
 - **Summary:** One-line description of what the function does
 - **Args:** Each parameter documented with type and meaning
 - **Returns:** What the function outputs
 - **Example:** Real usage showing input → output (concrete, searchable documentation)
- **IDE Integration:** Python's `help()` function and IDE tooltips display this automatically

6. Removed Intermediate Variable `c`

- **Original problem:** Extra variable clutters the code
- **How it helps:** Direct return is simpler, more efficient, and easier to follow. One less thing to mentally track
- **Pythonic:** Idiomatic Python avoids unnecessary intermediate variables

7. Fixed Indentation

- **Original problem:** `c=a+b` had inconsistent spacing
- **How it helps:** Proper 4-space indentation (PEP 8 standard) makes code visually consistent and machine-parseable

8. Added Spacing Around Operators

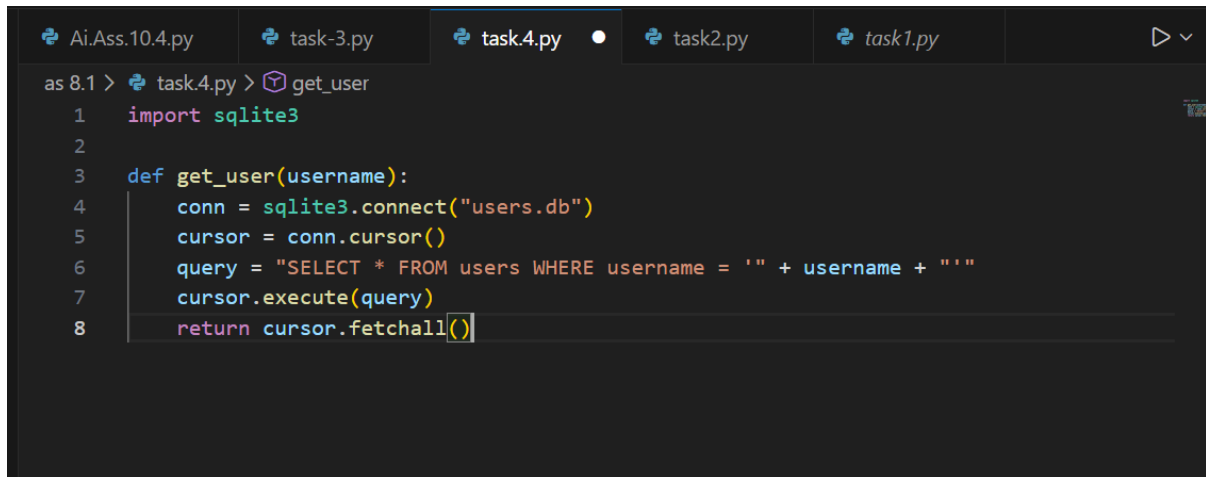
- **Original problem:** `a+b` and `a,b` lacked spaces

- **How it helps:** first number + second number is easier on the eyes.

Task-4: Secure Coding and Reliability Review

Prompt: Identify security vulnerabilities in this Python database code.

Refactor using parameterized queries. Add input validation. Add proper exception handling. Explain the security improvements



```
as 8.1 > task.4.py > get_user
1  import sqlite3
2
3  def get_user(username):
4      conn = sqlite3.connect("users.db")
5      cursor = conn.cursor()
6      query = "SELECT * FROM users WHERE username = '" + username + "'"
7      cursor.execute(query)
8      return cursor.fetchall()
```

```
as 8.1 > task4.py > get_user
1 import sqlite3

2 def get_user(username):
3     conn = sqlite3.connect("users.db")
4     cursor = conn.cursor()
5     query = "SELECT * FROM users WHERE username = " + username + ""
6     cursor.execute(query)
7     return cursor.fetchall()

8 from typing import List, Tuple, Optional
9
10 def validate_username(username: str) -> bool:
11     """
12     Validate username input.
13
14     Args:
15         username (str): The username to validate.
16
17     Returns:
18         bool: True if valid, False otherwise.
19     """
20     if not isinstance(username, str):
21         return False
22     if not username or len(username) > 50:
23         return False
24     # Allow only alphanumeric, underscore, and hyphen
25     return all(c.isalnum() or c in '_' '-' for c in username)
26
27 def get_user(username: str) -> Optional[List[Tuple]]:
28     """
29     Retrieve user information by username using parameterized query.
30
31     Args:
32         username (str): The username to search for.
33
34     Returns:
35         Optional[List[Tuple]]: List of user records if found, None if error occurs.
36
37     Raises:
38         ValueError: If username fails validation.
39     """
```

```
as 8.1 > task4.py > get_user
23 def get_user(username: str) -> Optional[List[Tuple]]:
24     Returns:
25         Optional[List[Tuple]]: List of user records if found, None if error occurs.
26
27     Raises:
28         ValueError: If username fails validation.
29     """
30     # Input validation
31     if not validate_username(username):
32         raise ValueError("Invalid username format. Must be alphanumeric, underscore, or hyphen, max 50 chars.")
33
34     conn = None
35     try:
36         # Connect to database
37         conn = sqlite3.connect("users.db")
38         cursor = conn.cursor()
39
40         # Use parameterized query (placeholder) - prevents SQL injection
41         query = "SELECT * FROM users WHERE username = ?"
42         cursor.execute(query, (username,))
43
44         results = cursor.fetchall()
45         return results if results else None
46
47     except sqlite3.DatabaseError as e:
48         print(f"Database error occurred: {e}")
49         return None
50     except sqlite3.OperationalError as e:
51         print(f"Database connection error: {e}")
52         return None
53     except Exception as e:
54         print(f"Unexpected error: {e}")
55         return None
56     finally:
57         # Always close the connection
58         if conn:
59             conn.close()
```

Explanation: Security Vulnerabilities & Fixes

Original Code - 4 Critical Issues:

1. **SQL Injection** - Attacker input concatenated directly into query
 - "SELECT * FROM users WHERE username = '" + username + "'"
 - Attack: ' OR '1'='1 returns all users
 2. **No Input Validation** - Accepts any characters, no length limit
 - Allows special characters like ', ;, --
 3. **No Exception Handling** - App crashes if database fails
 - No try-except blocks, connections never close
 4. **Resource Leak** - Connections never closed
 - 100 requests = 100 open connections = Database overload
-

Refactored Code - 4 Security Fixes:

1. **Parameterized Query** - User input treated as DATA, not SQL code
 - query = "SELECT * FROM users WHERE username = ?"
 - cursor.execute(query, (username,))
 - Special characters automatically escaped, injection impossible
2. **Input Validation** - Whitelist only safe characters (alphanumeric, _, -)
 - Rejects quotes, semicolons, comments before reaching database

- Max 50 characters, no empty strings

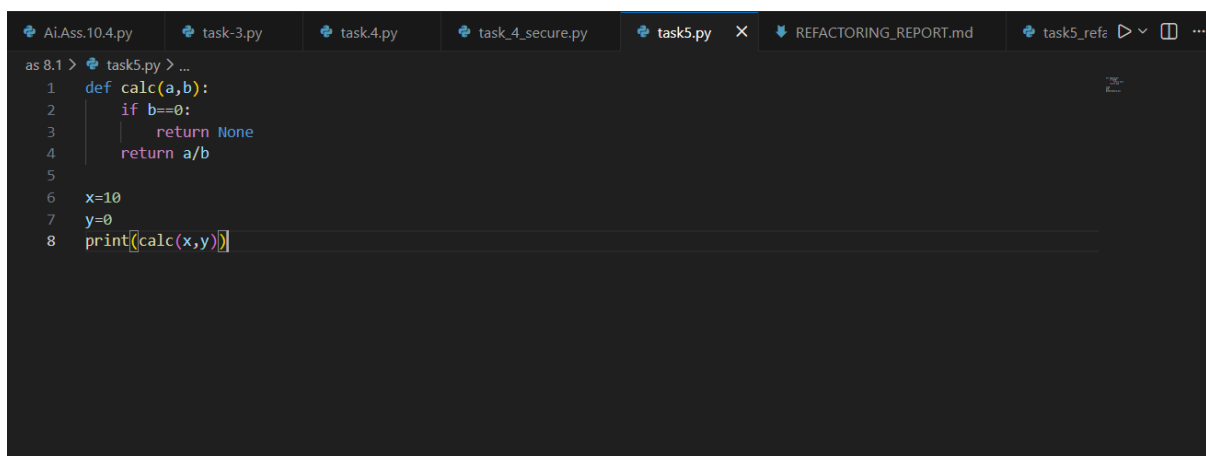
3. **Exception Handling** - Catches database errors gracefully

- try-except blocks handle connection failures
- App stays running, returns None instead of crashing

4. **Resource Management** - finally block always closes connections

- Prevents connection exhaustion
- Ensures cleanup even if error occurs.

Task-5: AI-Based Automated Code Review Report



```
as 8.1 > task5.py > ...
1  def calc(a,b):
2      if b==0:
3          return None
4      return a/b
5
6  x=10
7  y=0
8  print(calc(x,y))
```

Prompt: Analyze the following poorly written Python script and generate a structured code review report evaluating code readability, naming conventions, formatting and style consistency, error handling, documentation quality, and maintainability. Identify quality issues, potential risk areas, and code smells, and provide improvement suggestions. Optionally include an improved version of the code.

```

Ai.Ass.10.4.py task-3.py task-4.py task_4_secure.py task5.py SUMMARY.md PRODUCTION_TR/
as 8.1 > task5.py > ...
1 def divide_numbers(dividend, divisor):
2     """
3     Divides two numbers and returns the result.
4
5     Parameters:
6         dividend (int or float): The number to be divided.
7         divisor (int or float): The number to divide by.
8
9     Returns:
10        float: The division result.
11
12    Raises:
13        ZeroDivisionError: If divisor is zero.
14        TypeError: If inputs are not numeric.
15    """
16    if not isinstance(dividend, (int, float)) or not isinstance(divisor, (int, float)):
17        raise TypeError("Both inputs must be numeric.")
18
19    if divisor == 0:
20        raise ZeroDivisionError("Division by zero is not allowed.")
21
22    return dividend / divisor
23
24
25 try:
26     result = divide_numbers(10, 2)
27     print(result)
28 except Exception as error:
29     print("Error:", error)
```